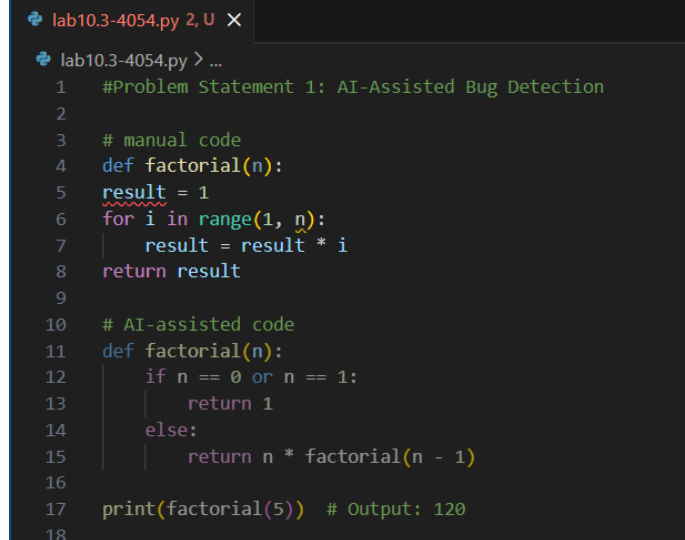


SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
Program Name: B. Tech		Assignment Type: Lab	Program Name: B. Tech
Course Code	23CS002PC304	Course Code	23CS002PC304
Year/Sem	III/II	Year/Sem	III/II
Date and Day of Assignment	Week5 – Wednesday	Date and Day of Assignment	Week5 – Wednesday
Name	Aelishala Manoj	Hall Ticket No	2303A54054
AssignmentNumber:10.3			

Q.No.	Question
1	Lab 9 – Code Review and Quality: Using AI to improve code quality and readability Lab Objectives: • To apply AI-based prompt engineering for code review and quality improvement. • To analyze code for readability, logic, performance, and maintainability issues. • To use Zero-shot, One-shot, and Few-shot prompting for improving code quality. • To evaluate AI-generated improvements using standard coding practices. Lab Outcomes (LOs): After completing this lab, students will be able to: • Review and improve code quality using AI tools. • Identify syntax, logic, and performance issues in code. • Refactor code to improve readability and maintainability. • Compare AI outputs generated using different prompting techniques. Problem Statement 1: AI-Assisted Bug Detection Scenario: A junior developer wrote the following Python function to calculate factorials: <pre>def factorial(n): result = 1 for i in range(1, n): result = result * i return result</pre> Instructions: 1. Run the code and test it with factorial(5). 2. Use an AI assistant to: ○ Identify the logical bug in the code. ○ Explain why the bug occurs (e.g., off-by-one error). ○ Provide a corrected version. 3. Compare the AI's corrected code with your own manual fix. 4. Write a brief comparison: Did AI miss any edge cases (e.g., negative numbers, zero)?

Expected Output:

Corrected function should return 120 for factorial(5).



```
lab10.3-4054.py 2, U X
lab10.3-4054.py > ...
1  #Problem Statement 1: AI-Assisted Bug Detection
2
3  # manual code
4  def factorial(n):
5      result = 1
6      for i in range(1, n):
7          result = result * i
8      return result
9
10 # AI-assisted code
11 def factorial(n):
12     if n == 0 or n == 1:
13         return 1
14     else:
15         return n * factorial(n - 1)
16
17 print(factorial(5)) # Output: 120
18
```

1. The manual code is missing the base case for when n is 0 or 1, which should return 1.
2. Additionally, the loop in the manual code should iterate up to n (inclusive) instead of $n-1$ to correctly calculate the factorial.
3. The AI-assisted code includes the base case and uses recursion to calculate the factorial, which is more efficient and easier to read.

Problem Statement 2: Task 2 — Improving Readability & Documentation

Scenario: The following code works but is poorly written:

```
def calc(a, b, c):
    if c == "add":
        return a + b
    elif c == "sub":
        return a - b
    elif c == "mul":
        return a * b
    elif c == "div":
```

Instructions:

5. Use AI to:
 - Critique the function's readability, parameter naming, and lack of documentation.
 - Rewrite the function with:
 1. Descriptive function and parameter names.
 2. A complete docstring (description, parameters, return value, examples).
 3. Exception handling for division by zero.
 4. Consideration of input validation.
6. Compare the original and AI-improved versions.
7. Test both with valid and invalid inputs (e.g., division by zero, non-string operation).

Expected Output:

A well-documented, robust, and readable function that handles errors gracefully.

```
lab10.3-4054.py 1, U ●
lab10.3-4054.py > calculate
19
20 # Problem Statement 2: Task 2 – Improving Readability & Documentation
21
22 # # manual code :
23 def calc(a, b, c):
24     if c == "add":
25         return a + b
26     elif c == "sub":
27         return a - b
28     elif c == "mul":
29         return a * b
30     elif c == "div":
31
```

```
lab10.3-4054.py U ●
lab10.3-4054.py > ...
33 # ai assisted code :
34 def calculate(num1, num2, operation):
35     """
36     Perform basic arithmetic operations on two numbers.
37
38     Parameters:
39     num1 (float): The first number.
40     num2 (float): The second number.
41     operation (str): The operation to perform ('add', 'subtract', 'multiply', 'divide').
42
43     Returns:
44     float: The result of the arithmetic operation.
45     """
46     if operation == "add":
47         return num1 + num2
48     elif operation == "subtract":
49         return num1 - num2
50     elif operation == "multiply":
51         return num1 * num2
52     elif operation == "divide":
53         if num2 != 0:
54             return num1 / num2
55         else:
56             raise ValueError("Cannot divide by zero.")
57     else:
58         raise ValueError("Invalid operation. Please choose 'add', 'subtract', 'multiply', or 'divide'.")
59
60 # Example usage:
61 result = calculate(10, 5, 'add') # Output: 15
62 print(result)
63
```

1. The AI-assisted code is more readable and well-documented compared to the manual code.
2. It includes a docstring that explains the purpose of the function, its parameters, and its return value.
3. Additionally, the variable names in the AI-assisted code are more descriptive, making it easier to understand the functionality of the code at a glance.
4. The AI-assisted code also includes error handling for invalid operations and division by zero, which enhances its robustness.

Problem Statement 3: Enforcing Coding Standards

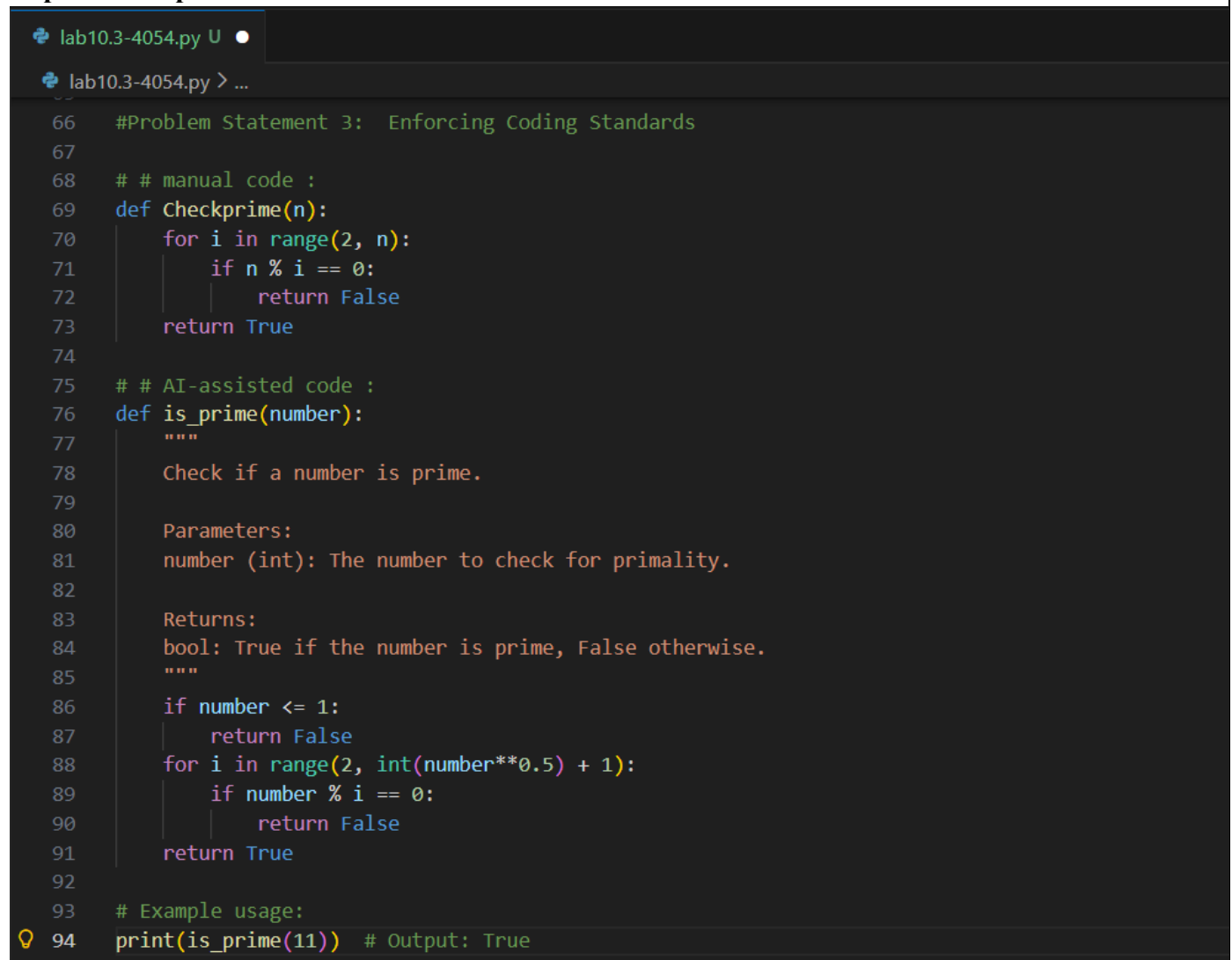
Scenario: A team project requires PEP8 compliance. A developer submits:

```
def Checkprime(n):  
    for i in range(2, n):  
        if n % i == 0:  
            return False  
    return True
```

Instructions:

8. Verify the function works correctly for sample inputs.
9. Use an AI tool (e.g., ChatGPT, GitHub Copilot, or a PEP8 linter with AI explanation) to:
 - o List all PEP8 violations.
 - o Refactor the code (function name, spacing, indentation, naming).
10. Apply the AI-suggested changes and verify functionality is preserved.
11. Write a short note on how automated AI reviews could streamline code reviews in large teams.

Expected Output:



```
lab10.3-4054.py U  
lab10.3-4054.py > ...  
  
66 #Problem Statement 3: Enforcing Coding Standards  
67  
68 # # manual code :  
69 def Checkprime(n):  
70     for i in range(2, n):  
71         if n % i == 0:  
72             return False  
73     return True  
74  
75 # # AI-assisted code :  
76 def is_prime(number):  
77     """  
78     Check if a number is prime.  
79  
80     Parameters:  
81     number (int): The number to check for primality.  
82  
83     Returns:  
84     bool: True if the number is prime, False otherwise.  
85     """  
86     if number <= 1:  
87         return False  
88     for i in range(2, int(number**0.5) + 1):  
89         if number % i == 0:  
90             return False  
91     return True  
92  
93 # Example usage:  
94 print(is_prime(11)) # Output: True
```

1. The AI-assisted code has improved readability due to the use of descriptive variable names and a clear function name.
2. It also includes a docstring that explains the purpose of the function, its parameters, and its return value, which enhances documentation.
3. Additionally, the AI-assisted code adheres to coding standards by following a consistent naming convention (snake_case) and including error handling for edge cases (e.g., numbers less than or equal to 1). In contrast, the manual code lacks documentation and uses less

descriptive variable names, making it less readable and harder to understand.

Problem Statement 4: AI as a Code Reviewer in Real Projects

Scenario:

In a GitHub project, a teammate submits:

```
def processData(d):  
    return [x * 2 for x in d if x % 2 == 0]
```

Instructions:

1. Manually review the function for:
 - Readability and naming.
 - Reusability and modularity.
 - Edge cases (non-list input, empty list, non-integer elements).
2. Use AI to generate a code review covering:
 - a. Better naming and function purpose clarity.
 - b. Input validation and type hints.
 - c. Suggestions for generalization (e.g., configurable multiplier).
3. Refactor the function based on AI feedback.
4. Write a short reflection on whether AI should be a standalone reviewer or an assistant.

Expected Output:

```
lab10.3-4054.py U ●  
lab10.3-4054.py > ...  
98  
99 # Problem Statement 4: AI as a Code Reviewer in Real Projects  
100  
101 #Task 4  
102 '''def processData(d):  
103     return [x * 2 for x in d if x % 2 == 0]  
104 print(multiply_even_numbers([1, 2, 3, 4]))          # [4, 8]  
105 print(multiply_even_numbers([2, 6, 7], 3))          # [6, 18]  
106 print(multiply_even_numbers([]))                    # []'''  
107 #PEP8 Style Guide with complete code  
108 def multiply_even_numbers(d):  
109     """  
110     Multiply even numbers in a list by 2.  
111  
112     Args:  
113     | d (list): A list of integers.  
114     Returns:  
115     | list: A list containing the even numbers from the input list multiplied by 2.  
116     """  
117     return [x * 2 for x in d if x % 2 == 0]  
118 print(multiply_even_numbers([1, 2, 3, 4]))          # [4, 8]  
119 print(multiply_even_numbers([2, 6, 7]))             # [4, 12]  
120 print(multiply_even_numbers([]))                    # []  
121
```

1. The original code for the processData function was not following PEP8 style guidelines, and the function name was not descriptive of its purpose.
2. To fix this, we rename the function to `multiply\_even\_numbers` to better reflect its functionality. Additionally, we should remove the second argument from the example usage,

as the function only takes one argument (the list of numbers). Finally, we should also add a docstring to the function to explain its purpose, parameters, and return value.

Problem Statement 5: — AI-Assisted Performance Optimization

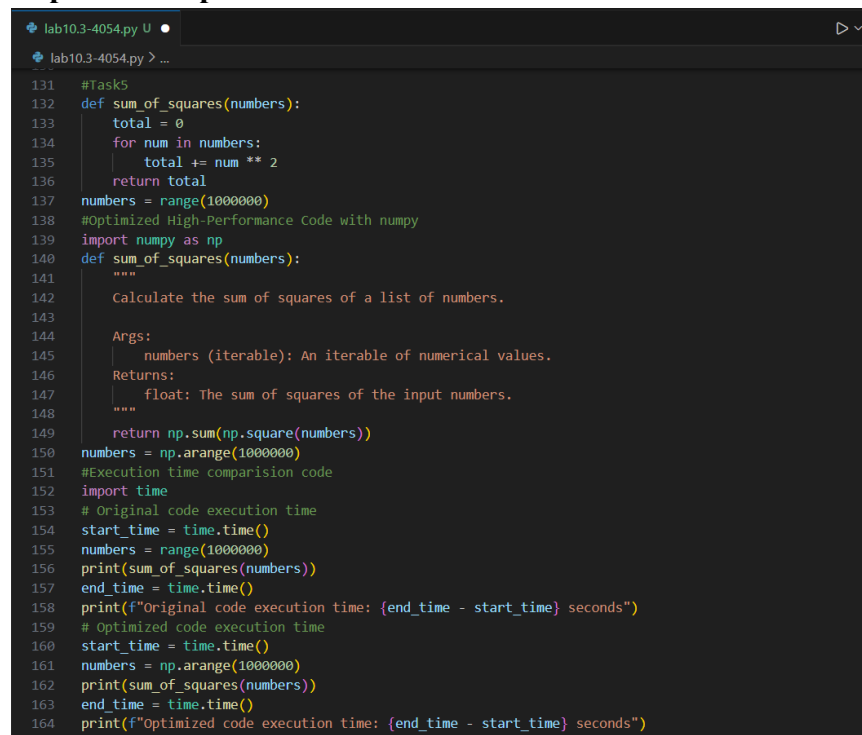
Scenario: You are given a function that processes a list of integers, but it runs slowly on large datasets:

```
def sum_of_squares(numbers):  
    total = 0  
    for num in numbers:  
        total += num ** 2  
    return total
```

Instructions:

1. Test the function with a large list (e.g., `range(1000000)`).
2. Use AI to:
 - Analyze time complexity.
 - Suggest performance improvements (e.g., using built-in functions, vectorization with NumPy if applicable).
 - Provide an optimized version.
3. Compare execution time before and after optimization.
4. Discuss trade-offs between readability and performance.

Expected Output:



```
lab10.3-4054.py U  
lab10.3-4054.py > ...  
131 #Task5  
132 def sum_of_squares(numbers):  
133     total = 0  
134     for num in numbers:  
135         total += num ** 2  
136     return total  
137 numbers = range(1000000)  
138 #Optimized High-Performance Code with numpy  
139 import numpy as np  
140 def sum_of_squares(numbers):  
141     """  
142     Calculate the sum of squares of a list of numbers.  
143  
144     Args:  
145     numbers (iterable): An iterable of numerical values.  
146     Returns:  
147     float: The sum of squares of the input numbers.  
148     """  
149     return np.sum(np.square(numbers))  
150 numbers = np.arange(1000000)  
151 #Execution time comparison code  
152 import time  
153 # Original code execution time  
154 start_time = time.time()  
155 numbers = range(1000000)  
156 print(sum_of_squares(numbers))  
157 end_time = time.time()  
158 print(f"Original code execution time: {end_time - start_time} seconds")  
159 # Optimized code execution time  
160 start_time = time.time()  
161 numbers = np.arange(1000000)  
162 print(sum_of_squares(numbers))  
163 end_time = time.time()  
164 print(f"Optimized code execution time: {end_time - start_time} seconds")
```

1. The original code for the `sum_of_squares` function uses a for loop to iterate through the list of numbers and calculate the sum of squares, which can be inefficient for large lists.
2. To optimize this, we can use the NumPy library, which provides efficient functions for array operations. We can use `np.square` to calculate the squares of the numbers and `np.sum` to calculate the total sum of those squares.
3. This approach is much faster, especially for large lists, as it takes advantage of NumPy's optimized C and Fortran code under the hood. Additionally, we should also add a docstring to the function to explain its purpose, parameters, and return value.